

Relatório de Auditoria de Segurança — Central NVR WiFi

Auditoria Estática de Vulnerabilidades, Controle de Acesso, Isolamento e Riscos de Rede

Projeto Auditado:	Central NVR WiFi para Linux (Desktop Client & NVR)
Versão / Release:	v1.0.0 (Python 3.10+ / PySide6 / PyAV / OpenCV / XDG)
Data da Auditoria:	29 de Agosto de 2026
Escopo Auditado:	src/central_nvr/*, packaging/*, scripts/*, install_ubuntu.sh
Classificação:	DOCUMENTO CONFIDENCIAL / SEGURANÇA TÉCNICA

Nota Metodológica e Mapeamento da Stack

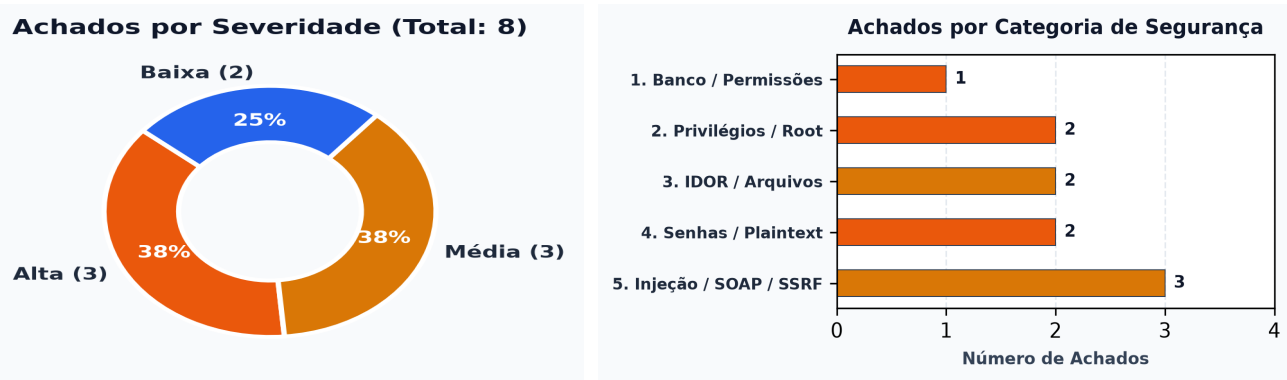
Esta auditoria analisou minuciosamente o código-fonte da aplicação desktop **Central NVR WiFi**. Como trata-se de um software nativo Linux cliente/NVR em Python com interface PySide6 e integração a protocolos de CFTV (ONVIF WS-Discovery, RTSP, SOAP), cada uma das cinco categorias clássicas de segurança foi adaptada ao ecossistema real:

Categoria Padrão	Equivalente Mapeado na Stack Central NVR WiFi
1. Banco sem Tranca	Permissões de arquivos e diretórios de persistência XDG (settings.json, devices.json) e isolamento multiusuário Linux.
2. Permissão no Navegador	Privilégios de execução no sistema operacional (scripts install_ubuntu.sh, pacotes postinst executados como root, permissão de arquivos em /opt).
3. IDOR / Acesso a Objetos	Manipulação de caminhos em nomes de arquivos, snapshots, gravações e identificadores de câmera (Path Traversal em salvamento e execução de handlers de arquivo).
4. Chaves Expostas (Hardcode)	Armazenamento em texto claro de senhas de câmeras IP, credenciais de conexão NVR, tokens de autenticação e falta de codificação em URIs.
5. Inputs / Injeção (XSS)	Injeção de XML/SOAP em chamadas ONVIF PTZ/Media, blind SSRF em endpoints XAddr e parsing inseguro de XML multicast com xml.etree.

1. Resumo Executivo

A auditoria identificou um total de **8 achados de segurança verificados no código real** (0 Críticos, 3 Altos, 3 Médios e 2 Baixos). Não foram identificadas vulnerabilidades de injeção de comandos de shell direta, graças ao uso consistente de subprocess.Popen e subprocess.run sem shell=True. No entanto, destacam-se riscos centrais associados ao **armazenamento de senhas em texto puro** em devices.json, **permissões permissivas no diretório de configuração do usuário**, **atribuição de propriedade insegura (chown) no instalador root** e **interpolação não sanitizada de tokens em envelopes SOAP ONVIF**.

CRÍTICA 0 achados	ALTA 3 achados	MÉDIA 3 achados	BAIXA 2 achados	PONTOS FORTES 4 verificados
-----------------------------	--------------------------	---------------------------	---------------------------	---------------------------------------



2. Avaliação de Postura: Pontos Fortes e Riscos Centrais

Pontos Fortes Verificados no Código:

SEGURO	Ofuscação de Credenciais em Logs: stream_worker.py:69-74 mascara senhas RTSP com regex antes de emitir logs.
SEGURO	Execução Segura de Subprocessos: playback_view.py:224 e diagnose_camera.py:85 usam listas sem shell=True.
SEGURO	Digest WS-Security OASIS: onvif_client.py:54-91 gera PasswordDigest SHA-1 com Nonce sem enviar senha em claro no SOAP.
SEGURO	Isolamento de Threads: Rede e decodificação rodam em QThreads dedicadas, evitando travamentos da GUI.

Riscos Centrais Identificados:

- **Exposição de Credenciais Locais:** Senhas gravadas em texto puro em devices.json com permissões padrão do SO (0644).
- **Privilégios de Sistema em Instalação:** install_ubuntu.sh entrega /opt/central-nvr a usuário comum enquanto wrapper roda como root.
- **Manipulação SOAP/ONVIF:** Interpolação direta de tokens não sanitizados em envelopes SOAP e aceitação de XAddr arbitrários sem validação.

3. Tabela Detalhada de Achados de Segurança

Sev.	ID / Categoria	Arquivo : Linhas	Descrição do Problema e Causa Raiz
ALTA	SEC-01 1. Banco / Isolamento	src/central_nvr/core/config.py L23, L107-108, L116-117	Permissões Inseguras em Arquivos de Credenciais: devices.json criado com umask padrão (0644), permitindo leitura de senhas por outros usuários locais.
ALTA	SEC-02 4. Chaves / Plaintext	src/central_nvr/core/camera.py L138, L159	Armazenamento de Senha em Texto Puro: O campo password é serializado diretamente sem criptografia em repouso ou integração com SecretService/Keyring.
ALTA	SEC-03 2. Privilégios / Root	install_ubuntu.sh L74, L81	chown Inseguro em Diretório de Sistema: /opt/central-nvr é entregue ao CURRENT_USER enquanto /usr/local/bin/central-nvr executa seu Python, abrindo vetor de LPE.
MÉDIA	SEC-04 2. Privilégios / Root	packaging/debian/postinst L8-11	pip install como Root no postinst: Script deb instala pacotes PyPI via rede como root sem pinning ou hash verification, suscetível a MITM/supply-chain.
MÉDIA	SEC-05 5. Injeção / SOAP	src/central_nvr/core/onvif_client.py L246, L277, L343	Injeção de XML/SOAP em Tokens ONVIF: ProfileToken e PresetToken interpolados sem escape XML em envelopes SOAP, permitindo adulteração de estrutura.
MÉDIA	SEC-06 5. Injeção / SSRF	src/central_nvr/core/onvif_client.py L189-199 src/central_nvr/scanner/parser.py L55-64	SSRF em XAddr de Capacidades ONVIF: Cliente redireciona requisições SOAP autenticadas para URLs arbitrárias em <tt:XAddr> sem validar se o host corresponde ao IP da câmera.
MÉDIA	SEC-07 3. IDOR / Arquivos	src/central_nvr/ui/camera_view.py L384-386 src/central_nvr/ui/fullscreen_view.py L262-264	Path Traversal Potencial em Snapshots: O nome do snapshot concatena self.camera.id. Se o ID contiver ../, pode gravar fora da pasta snapshots/.
BAIXA	SEC-08 4. Credenciais / Scripts	scripts/diagnose_camera.py L63-64	Falta de URL-Encoding em Credenciais no Diagnóstico: Concatenação de senha na URL RTSP sem urllib.parse.quote, corrompendo conexão com caracteres especiais.

4. Plano de Ação e Recomendações Priorizadas

Prioridade P1 (Imediata)	1. Restringir Permissões de Arquivo: Ajustar save_devices() para gravar com 0600 (os.open com O_CREAT O_WRONLY e 0o600) e diretório com 0700. 2. Corrigir Permissões do Instalador: Em install_ubuntu.sh, manter /opt/central-nvr como root:root 755, instalando pacotes no build e evitando que usuários alterem os binários executados pelo wrapper global.
Prioridade P2 (Curto Prazo)	3. Escape de XML em Tokens ONVIF: Aplicar html.escape() em todos os parâmetros interpolados nos métodos de OnvifClient. 4. Sanitização de IDs de Câmeras: Aplicar re.sub(r'[^\a-zA-Z0-9_-]', '', camera.id) antes de usá-lo em nomes de arquivos de snapshot. 5. Validação de Host em XAddr (Anti-SSRF): Verificar se o host retornado em XAddr corresponde ao IP configurado da câmera.
Prioridade P3 (Médio Prazo)	6. Criptografia em Repouso / Keyring: Integrar com a biblioteca keyring do sistema operacional para armazenar senhas no cofre nativo (GNOME Keyring / KWallet). 7. Refatoração do Pacote Debian: Remover pip install do postinst; declarar dependências nativas no debian/control.

5. Issues Acionáveis para o GitHub (Parte 1)

Templates completos em Markdown prontos para abertura de issues no repositório:

Issue 1: Permissões Inseguras e Senhas em devices.json

```
--- ISSUE 1 ---
**Título:** [Segurança] Permissões inseguras e armazenamento em texto claro de senhas em devices.json
**Labels:** security, severity:high, bug, storage

### Descrição do Problema
O arquivo `~/config/central-nvr/devices.json` armazena senhas de câmeras IP e credenciais NVR em texto puro (plaintext) e é salvo com a máscara de permissões padrão do sistema (0644). Em estações de trabalho ou servidores Linux multiusuário, qualquer usuário local pode ler o arquivo e obter credenciais de toda a rede de monitoramento.

### Evidência
- `src/central_nvr/core/config.py:116-117`:
  ```python
 with open(self.devices_path, "w", encoding="utf-8") as f:
 json.dump(self.devices, f, indent=2, ensure_ascii=False)
  ```
- `src/central_nvr/core/camera.py:138`:
  ```python
 "password": self.password,
  ```

### Impacto
Exposição total de senhas de CFTV para processos e usuários não privilegiados na mesma máquina.

### Sugestão de Correção
1. Criar o diretório com `0700` e arquivos com `0600` utilizando `os.open(path, os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o600)`.
2. Integrar suporte opcional ao `keyring` do sistema para proteção de senhas.

### Critérios de Aceite
- [ ] `devices.json` e `settings.json` são criados com permissão estrita `0600`.
- [ ] O diretório `~/config/central-nvr` possui permissão `0700`.
- [ ] Testes automatizados confirmam permissão octal em sistemas POSIX.
--- FIM ISSUE 1 ---
```

Issue 2: Elevação de Privilégios no Instalador e Pacote Debian

```
--- ISSUE 2 ---
**Título:** [Segurança] Vetor de elevação de privilégios por chown inseguro em install_ubuntu.sh e pip no postinst
**Labels:** security, severity:high, packaging, installer

### Descrição do Problema
O instalador `install_ubuntu.sh` executa como root e transfere a posse de `/opt/central-nvr` para o `CURRENT_USER` desprivilegiado, enquanto `/usr/local/bin/central-nvr` executa o interpretador desse diretório. Se o administrador (root) ou outro usuário executar o binário global, código injetado por processos do usuário comum será executado com privilégios elevados. Além disso, `packaging/debian/postinst` executa `pip install` arbitrário pela rede como root.

### Evidência
- `install_ubuntu.sh:74, 81`:
  ```bash
 chown -R "${CURRENT_USER}:${CURRENT_USER}" "${INSTALL_DIR}"
 ...
 exec /opt/central-nvr/venv/bin/python3 -m central_nvr.app "$@"
  ```
- `packaging/debian/postinst:10`:
  ```bash
 /opt/central-nvr/venv/bin/pip install -q PySide6 opencv-python-headless av requests pillow numpy
  ```

### Impacto
Potencial elevação de privilégios local para root e risco de supply-chain durante a instalação do pacote deb.

### Sugestão de Correção
1. Manter `/opt/central-nvr` pertencente a `root:root` com permissão `755` (somente leitura para usuários normais).
2. Remover execução do `pip` no `postinst` e declarar dependências nativas no `debian/control`.

### Critérios de Aceite
- [ ] `/opt/central-nvr` não pertence a usuário desprivilegiado.
- [ ] O script `postinst` não executa download de pacotes via pip em tempo de instalação.
--- FIM ISSUE 2 ---
```

5. Issues Acionáveis para o GitHub (Parte 2)

Templates completos em Markdown prontos para abertura de issues no repositório:

Issue 3: Injeção de XML/SOAP e Blind SSRF em Endpoints ONVIF

```
--- ISSUE 3 ---
**Título:** [Segurança] Interpolação de XML não escapado em chamadas ONVIF PTZ/Media e risco de SSRF em XAddr
**Labels:** security, severity:medium, onvif, injection

### Descrição do Problema
O cliente `OnvifClient` interpola variáveis como `profile_token` e `preset_token` diretamente em envelopes SOAP sem sanitização XML, abrindo vetor para injeção de tags XML SOAP. Adicionalmente, o cliente aceita URLs `XAddr` arbitrárias retornadas por respostas ONVIF e envia requisições autenticadas com WS-Security para esses destinos sem validação de host.

### Evidência
- `src/central_nvr/core/onvif_client.py:246`:
  ```python
 {token}
  ```
- `src/central_nvr/core/onvif_client.py:191`:
  ```python
 self.media_service_url = media_match.group(1).strip()
  ```

### Impacto
Adulteração de mensagens SOAP e envio de requisições autenticadas para servidores internos arbitrários (SSRF).

### Sugestão de Correção
1. Aplicar `html.escape(token)` em todos os parâmetros inseridos em templates XML.
2. Validar que o host de qualquer `XAddr` seja idêntico ao IP configurado da câmera.

### Critérios de Aceite
- [ ] Todos os tokens em `onvif_client.py` passam por escape XML.
- [ ] Validação de host rejeita `XAddr` divergente do IP da câmera.
--- FIM ISSUE 3 ---
```

Issue 4: Sanitização de Nomes de Snapshots e Encoding de URL

```
--- ISSUE 4 ---
**Título:** [Segurança] Path Traversal potencial na nomenclatura de snapshots e falta de encoding de URL no script de diagnóstico
**Labels:** security, severity:low, ui, sanitization

### Descrição do Problema
O método de captura de snapshot concatena diretamente `self.camera.id` no caminho do arquivo sem sanitização de caracteres especiais ou travessia de diretório (`../`). No script `scripts/diagnose_camera.py`, as credenciais são concatenadas sem `urllib.parse.quote`.

### Evidência
- `src/central_nvr/ui/camera_view.py:385-386`:
  ```python
 filename = f"snap_{self.camera.id}_{ts}.jpg"
 filepath = str(snap_dir / filename)
  ```
- `scripts/diagnose_camera.py:63-64`:
  ```python
 auth = f"{user}:{password}@" if password else ""
 url = f"rtsp://{auth}{ip}:554/onvif1"
  ```

### Impacto
Possibilidade de gravação de arquivos fora do diretório pretendido caso o ID da câmera seja manipulado e falha de autenticação com senhas complexas.

### Sugestão de Correção
1. Sanitizar `camera.id` com `re.sub(r'^a-zA-Z0-9_-|', '_', self.camera.id)`.
2. Aplicar `urllib.parse.quote(password)` na construção de URIs RTSP no script de diagnóstico.

### Critérios de Aceite
- [ ] Snapshots gravados contêm apenas caracteres seguros no nome do arquivo.
- [ ] Script de diagnóstico codifica corretamente caracteres especiais em senhas.
--- FIM ISSUE 4 ---
```